School Management System - Complete Project Guide

Generated on: May 18, 2026

1. Project Overview

This project is a full-stack School Management System built with a React frontend and an Express/MongoDB backend. The main goal is to manage a school workflow from one web application:

- Admin can manage students, faculty, classes, bulk uploads, reset credentials, and chat.
- Teacher can view assigned classes, mark attendance, upload marks, send notices, use bulk uploads, and chat.
- Student can view attendance, marks, notices, profile information, and chat with the assigned teacher.

The application uses role-based access control. Each user logs in with an email and password, receives a JWT token, and can access only the routes allowed for their role.

2. Technology Stack

Frontend:

- React 19
- React Router DOM
- Axios
- Vite / Rolldown Vite
- Tailwind CSS

Backend:

- Node.js
- Express 5
- MongoDB
- Mongoose
- JWT authentication
- bcryptjs password hashing
- Multer for file upload
- XLSX and CSV parser utilities
- Nodemailer for email credentials

Database:

- MongoDB with Mongoose models.

3. High-Level Architecture

The project has two main folders:

- frontend: React user interface.
- backend: Express API server and MongoDB models.

Basic request flow:

1. User opens the React app.
2. User logs in through /api/auth/login.
3. Backend validates credentials and returns a JWT token.
4. Frontend stores the token in localStorage.
5. Axios sends the token in the Authorization: Bearer <token> header.
6. Backend middleware verifies the token and role.
7. Controller performs database operations and returns an ApiResponse.

4. User Roles

Admin

Admin is the highest-control role. Admin features include:

- Dashboard counts for total teachers, students, and classes.
- Create, edit, view, search, and delete faculty profiles.
- Create, edit, view, search, and delete student profiles.
- Create, edit, assign, and delete classes.
- Bulk upload students and faculty from CSV/Excel.
- Resend reset credentials to users by email.
- Chat with teachers and students.

Teacher

Teacher features include:

- View own teacher profile.
- View assigned classes.
- View students inside assigned classes.
- Mark daily attendance.
- Upload subject-wise marks.
- Bulk upload marks.
- Bulk upload attendance.
- Send notices to assigned students.
- View sent notices.
- Chat with admin and assigned students.

Student

Student features include:

- View own profile and assigned class.
- View attendance records.
- View marks records.
- View notices sent by teacher.
- Chat with assigned teacher.

5. Frontend Structure

Important frontend files:

- frontend/src/App.jsx: Defines all application routes.
- frontend/src/api/axiosInstance.js: Central Axios instance with API base URL and token interceptor.
- frontend/src/context/AuthContext.jsx: Stores logged-in user data and login/logout behavior.
- frontend/src/components/ProtectedRoute.jsx: Protects pages based on user role.
- frontend/src/components/Layout.jsx: Main dashboard layout shell.
- frontend/src/components/Input.jsx: Reusable input component with password show/hide support.
- frontend/src/components/Button.jsx: Reusable button component.
- frontend/src/components/Table.jsx: Reusable table component.
- frontend/src/utils/formValidation.js: Shared frontend validation helpers.

Admin pages:

- AdminDashboard.jsx: Admin metrics and navigation.
- Teachers.jsx: Faculty create/edit/view/delete/search.
- Students.jsx: Student create/edit/view/delete/search.
- Classes.jsx: Class create/edit/delete and faculty assignment.
- BulkUpload.jsx: Upload CSV/Excel for students and faculty.

Teacher pages:

- TeacherDashboard.jsx: Teacher landing dashboard.
- TeacherClasses.jsx: Assigned class list.
- Attendance.jsx: Manual attendance marking.
- Marks.jsx: Manual marks upload.
- BulkMarksUpload.jsx: Bulk marks upload.
- BulkAttendanceUpload.jsx: Bulk attendance upload.
- Notices.jsx: Send and view student notices.

Student pages:

- StudentDashboard.jsx: Student landing dashboard.
- MyAttendance.jsx: Attendance history.
- MyMarks.jsx: Marks history.
- Notices.jsx: Student notices.

Shared page:

- ChatCenter.jsx: Chat screen for role-based contacts and message thread.

6. Frontend Routes

Public routes:

- #/login: Login page.
- #/register: First admin registration page.

- #/change-password: Forced password change page for generated accounts.

Admin protected routes:

- #/admin
- #/admin/teachers
- #/admin/students
- #/admin/classes
- #/admin/bulk-upload
- #/admin/chat

Teacher protected routes:

- #/teacher
- #/teacher/classes
- #/teacher/attendance
- #/teacher/marks
- #/teacher/bulk-marks
- #/teacher/bulk-attendance
- #/teacher/notices
- #/teacher/chat

Student protected routes:

- #/student
- #/student/attendance
- #/student/marks
- #/student/notices
- #/student/chat

7. Backend Structure

Important backend files:

- backend/server.js: Loads environment variables, connects MongoDB, starts server.
- backend/src/app.js: Express app, CORS, JSON parsing, and route mounting.
- backend/src/config/db.js: MongoDB connection.
- backend/src/middlewares/auth.middleware.js: JWT token verification.
- backend/src/middlewares/role.middleware.js: Role authorization.
- backend/src/middlewares/upload.middleware.js: Multer memory upload, 3MB limit.
- backend/src/utils/ApiResponse.js: Standard API response wrapper.
- backend/src/utils/generatePassword.js: Temporary password generator.
- backend/src/utils/sendMail.js: Queues/sends emails.
- backend/src/utils/mailTransporter.js: SMTP transporter setup.
- backend/src/utils/emailTemplates.js: HTML email templates.
- backend/src/utils/parseFile.js: CSV/Excel upload parser entry.
- backend/src/utils/validation.js: Shared backend validation helpers.

8. Backend API Routes

Base URL:

- Local backend: http://localhost:8000/api
- Frontend Axios uses VITE_API_URL if provided, otherwise window.location.origin/api.

Auth APIs

- POST /api/auth/register: Register first admin or role user.
- POST /api/auth/login: Login and receive JWT token.

User APIs

- POST /api/user/change-password: Change password for logged-in user.

Admin APIs

- GET /api/admin/dashboard: Dashboard counts.
- POST /api/admin/teacher: Create teacher/faculty profile and account.
- GET /api/admin/teachers: List teachers.
- PUT /api/admin/teacher/:teacherId: Update teacher.
- DELETE /api/admin/teacher/:teacherId: Delete teacher and linked user data.
- POST /api/admin/student: Create student profile and account.
- GET /api/admin/students: List students.
- PUT /api/admin/student/:studentId: Update student.
- DELETE /api/admin/student/:studentId: Delete student and linked user data.
- POST /api/admin/class: Create class.
- GET /api/admin/classes: List classes with assigned faculty.
- PUT /api/admin/class/:classId: Update class.
- DELETE /api/admin/class/:classId: Delete class and related records.
- POST /api/admin/resend/:userId: Generate and email reset credentials.

Bulk APIs

- POST /api/bulk/students: Admin bulk upload for students.
- POST /api/bulk/teachers: Admin bulk upload for faculty.

Teacher APIs

- GET /api/teacher/me: Teacher profile.
- GET /api/teacher/classes: Classes assigned to logged-in teacher.
- GET /api/teacher/class/:classId/students: Students in an assigned class.
- POST /api/teacher/attendance: Save attendance.
- POST /api/teacher/marks: Save marks.
- GET /api/teacher/attendance-status: Today’s attendance status.
- POST /api/teacher/bulk/marks: Bulk marks upload.
- POST /api/teacher/bulk/attendance: Bulk attendance upload.
- GET /api/teacher/notices: Teacher’s sent notices.
- POST /api/teacher/notices: Send notice to a student.

Student APIs

- GET /api/student/me: Student profile.

- GET /api/student/attendance: Student attendance list.
- GET /api/student/marks: Student marks list.
- GET /api/student/notices: Student notices.

Chat APIs

- GET /api/chat/contacts: Role-based chat contacts.
- GET /api/chat/thread/:otherUserId: Message thread with another allowed user.
- POST /api/chat/send: Send message to an allowed user.

9. Database Models

User

Collection stores login accounts:

- name
- email
- password
- mustChangePassword
- passwordChangedAt
- role: ADMIN, TEACHER, or STUDENT

Password is hashed by a Mongoose pre-save hook using bcrypt.

Teacher

Teacher profile fields include:

- Linked user
- Subject and specialization details
- Phone and alternate phone
- DOB, gender, address, city, state, pincode
- Qualification, certifications, experience, designation
- Online teaching experience, devices, tech rating
- Demo readiness and declaration fields

Student

Student profile fields include:

- Linked user
- classId
- Roll number
- Phone and address
- DOB, gender, city, state, pincode
- Guardian name and guardian phone
- Admission number, section, previous school
- Medical notes, transport mode, admin notes

Class

Stores academic groups:

- name
- teacher

Attendance

Stores daily attendance:

- classId
- studentId
- date
- status: PRESENT or ABSENT
- markedBy

There is a unique index on classId + studentId + date, so duplicate attendance for the same day is prevented.

Mark

Stores marks:

- classId
- studentId
- subject
- marks
- maxMarks
- uploadedBy

Notice

Stores teacher-to-student notices:

- studentId
- teacherId
- title
- message
- status: UNREAD or READ

ChatMessage

Stores chat messages:

- sender
- recipient
- body

10. Authentication and Authorization

Authentication uses JWT:

1. User logs in.
2. Backend signs token using JWT_SECRET.

3. Frontend stores token in localStorage.
4. Axios sends token with every request.
5. protect middleware verifies token.
6. authorize middleware checks role.

Role protection examples:

- Admin APIs require ADMIN.
- Teacher APIs require TEACHER.
- Student APIs require STUDENT.
- Chat APIs require login, then controller restricts contact access.

11. Main Workflows

Admin creates a faculty account

1. Admin fills faculty form.
2. Frontend validates name, email, phone, pincode, and experience.
3. Backend validates again.
4. Backend creates User with role TEACHER.
5. Backend creates linked Teacher profile.
6. Temporary password is generated.
7. Credentials email is queued.
8. Teacher logs in and changes password if required.

Admin creates a student account

1. Admin fills student form.
2. Frontend validates name, email, phone, guardian phone, and pincode.
3. Backend validates again.
4. Backend creates User with role STUDENT.
5. Backend creates linked Student profile.
6. Credentials email is queued.

Teacher marks attendance

1. Teacher selects assigned class.
2. Students are loaded for that class.
3. Teacher marks PRESENT or ABSENT.
4. Backend verifies that class belongs to the teacher.
5. Attendance is upserted by class, student, and date.

Teacher uploads marks

1. Teacher selects assigned class.
2. Teacher enters subject and max marks.
3. Teacher enters marks per student.
4. Frontend validates marks range.
5. Backend verifies class ownership.
6. Marks are saved/upserted.

Student views data

1. Student logs in.
2. Student pages call /api/student/*.
3. Backend finds student profile using logged-in user ID.
4. Attendance, marks, notices, and profile data are returned.

12. Bulk Upload

Admin bulk upload supports:

- Students
- Faculty

Teacher bulk upload supports:

- Marks
- Attendance

Supported file types:

- CSV
- XLSX
- XLS

File upload limit:

- 3 MB

Student bulk template columns:

name,email,className,rollNo,phone,address,dob,gender,city,state,pincode,guardianName,guardianPhone,admissionNo,s

Faculty bulk template columns:

name,email,subject,phone,alternatePhone,dob,gender,address,city,state,pincode,qualification,specialization,certifications,e

Bulk upload returns a report:

- Total rows
- Created rows
- Failed rows
- Row-wise error reasons

13. Validation Rules

Frontend validation is in:

- frontend/src/utils/formValidation.js

Backend validation is in:

- backend/src/utils/validation.js

Important validation rules:

- Email must be valid.
- Phone numbers must contain exactly 10 digits.
- Guardian phone and alternate phone must also contain exactly 10 digits when provided.
- Pincode must contain exactly 6 digits when provided.
- Password must be at least 6 characters.
- Class name is required.
- Marks must be between 0 and max marks.
- Teacher can upload attendance/marks only for assigned classes.

Backend validation is important because direct API requests and bulk uploads should also be blocked from saving invalid data.

14. Email Credential System

When admin creates or resets a teacher/student account:

1. Backend generates a temporary password.
2. Password is saved after hashing.
3. Email template is prepared.
4. Mail is sent/queued using Nodemailer.

Environment variables used for email:

- SMTP_HOST
- SMTP_PORT
- SMTP_USER
- SMTP_PASS

Default SMTP host is Gmail (smtp.gmail.com) with common fallback ports.

15. Chat System

Chat is role-aware:

- Admin can chat with teachers and students.
- Teacher can chat with admin and assigned students.
- Student can chat with assigned teacher.

The backend calculates allowed contacts based on role and class assignment. This prevents users from opening unrelated chat threads.

16. Environment Variables

Backend .env example:

```
PORT=8000
MONGODB_URL=mongodb://127.0.0.1:27017/school-management
JWT_SECRET=your_secret_key
FRONTEND_URL=http://localhost:5173
CORS_ORIGIN=http://localhost:5173
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
```

```
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password
```

Frontend .env example:

```
VITE_API_URL=http://localhost:8000
```

17. How to Run Locally

Backend:

```
cd backend
npm install
npm run dev
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Default URLs:

- Frontend: http://localhost:5173
- Backend: http://localhost:8000

18. Build and Verification

Frontend build command:

```
cd frontend
npm run build
```

Backend syntax can be checked with:

```
cd backend
node --check src/controllers/admin.controller.js
```

Known lint note:

At the time of this guide, frontend build passes. The lint command still reports some older unrelated issues in files such as AuthContext.jsx, ChatCenter.jsx, StudentDashboard.jsx, and teacher/Notices.jsx.

19. Security Points

Current security features:

- JWT based authentication.
- Role based route protection.
- bcrypt password hashing.
- Teacher can access only assigned classes.
- Chat contact access is restricted by role and assignment.
- Validation exists on both frontend and backend.
- Upload size limit is set to 3 MB.

Recommended future security improvements:

- Add rate limiting for login.
- Add stricter server-side marks validation.
- Add refresh token or token expiry handling UX.
- Add audit logs for admin changes.
- Add password strength rules.
- Move more validation into Mongoose schema validators.

20. Future Improvements

Useful next improvements:

- Add downloadable reports for attendance and marks.
- Add student promotion workflow.
- Add fee management.
- Add timetable management.
- Add parent login role.
- Add notification read/unread update API.
- Add dashboard charts.
- Add automated backend tests.
- Add form-level inline error messages instead of only alerts.
- Add CSV error export after bulk upload.

21. Project Summary

This School Management System is a practical MERN-style project with real school workflows:

- Authentication
- Admin management
- Faculty management
- Student management
- Class assignment
- Attendance
- Marks
- Notices
- Bulk upload
- Email credentials
- Role-based chat

The project is organized clearly into frontend pages, reusable components, backend routes, controllers, models, middleware, and utilities. It is suitable for a college project, portfolio project, or as a base for a more complete school ERP system.